

Warm up with Python

CL249

Try the following

1.

```
print("Hello World")
```

2. To print a number or any variable

```
x = 9
```

```
print(f"x = {x}")
```

```
Print("x = ", x)
```

3. Boolean

```
4>3
```

Functions

A **function** is a reusable block of code that performs a single, specific task. It is like a recipe or an automated machine

- **Inputs (Parameters):** Ingredients you feed into the machine.
- **Process:** The instructions inside the machine that do the work.
- **Output (Return Value):** The finished product handed back to you.

```
def function_name(parameter1, parameter2):  
    # Code block (indented)  
    result = parameter1 + parameter2  
    return result
```

Functions

```
def greet(name):  
    print("Hello ", name)
```

To Run:

```
name = "Anita"  
greet(name)
```

More Examples

```
def greet_user(name):  
  
    #Generates a personalized greeting message.  
    greeting = f"Hello, {name}! Welcome to Python class."  
    return greeting
```

Calling the function:

```
message1 = greet_user("Aditi")  
message2 = greet_user("Tanmay")  
print(message1)  
print(message2)
```

Output: Hello, Aditi! Welcome to Python class.

Output: Hello, Tanmay! Welcome to Python class.

Conditions- Functions

Check if a number is divisible by 11

```
def check_eleven(x):  
    # set result to False  
    result = False  
    if (x%11)==0:  
        print(f'{x} is divisible by 11')  
        result = True  
    else:  
        print(f'{x} is not divisible by 11')  
    return result
```

To Run it:

```
x=22  
result = check_eleven(x)
```

Why Use Functions?

- **Avoid Repetition (DRY - Don't Repeat Yourself):** Write code once, use it as many times as you want.
- **Organization:** Break a complex program down into smaller, manageable pieces. Easier to debug if you are dealing with smaller pieces.
- **Readability:** Named functions act like self-documenting steps.

Drill 1.1

- Write a code to check if a number is even or odd. Use a function

Lists

Creating a list of fruits

```
fruits = ["apple", "banana", "cherry", "date"]
```

Accessing elements by index

```
first_item = fruits[0]
```

#Output: 'apple'

```
last_item = fruits[-1]
```

(negative index counts from the end)

Slicing a subset of the list [start:stop]

```
subset = fruits[1:3]    # ['banana', 'cherry']
```

Access to items in lists

```
numbers = [10, 20, 30, 40, 50]
# First item (Index 0)
first = numbers[0]
print(first) # Output: 10
# Third item (Index 2)
third = numbers[2]
print(third) # Output: 30
```

```
# Last item using negative index (-1
starts from the end)
last = numbers[-1]
print(last) # Output: 50
# Second to last item
second_last = numbers[-2]
print(second_last) # Output: 40
```

More examples: shorthand operations on all items in the list

```
numbers = [1, 2, 3, 4, 5, 6, 7, 8, 9, 10]
```

```
# Double every number
```

```
doubled = [x * 2 for x in numbers]
```

```
# Keep only even numbers
```

```
evens = [x for x in numbers if x%2 == 0 ]
```

```
print("Doubled:", doubled)
```

```
# Output: [2, 4, 6, 8, 10, 12, 14, 16, 18, 20]
```

```
print("Evens:", evens)
```

```
# Output: [2, 4, 6, 8, 10]
```

Drill 1.2

Given a list of scores, count how many students scored **70 or above**.

Sample scores

```
student_scores = [85, 62, 70, 95, 45, 78, 69, 100]
```

Convert between variable type

a = "12" :What type is *a* ?

To check type:

type(a)

Now change it to integer:

a = int(a)

Check again

Drill 1.3

name = "Alice"

age_str = "21"

gpa = 3.8

is_student = True

TODO: Convert age_str to an integer

TODO: Calculate age next year

TODO: Print the converted age and its type

Drill 1.4

Write a function `safe_divide(a, b)` that takes two numbers and returns a/b .

If b is 0, return "Cannot divide by zero" instead of throwing an error.

For loops

```
for i in range(0,5):  
    print(f"Iteration {i}")
```

```
for i in range(0,5):  
    print("Iteration ",i)
```

Drill 1.5: Write a function to print the multiplication table of 5

Drill 1.6

Sample dataset of Celsius recordings

```
celsius_readings = [12.5, -3.0, 0.0, 25.4, -10.2, 18.1]
```

1. Filter out any temperature that falls below freezing $< 0^{\circ}\text{C}$).
2. Convert the remaining temperatures to Fahrenheit using the formula:
3. $F = (C \times 1.8) + 32$
4. Return a new list containing the converted temperatures rounded to 1 decimal place.

Drill 1.7

Calculate e^x using series expansion-

$$1 + x + \frac{x^2}{2!} + \frac{x^3}{3!} + \dots$$

You need a function to calculate factorial $n!$ And a function for power

Plotting

```
import matplotlib.pyplot as plt
import numpy as np
```

```
#1. Generate x values (100
#evenly spaced numbers from -
#10 to 10)
```

```
x = np.linspace(-10, 10, 100)
```

```
# 2. Define the function
```

```
# $y = 2x + 3$ 
```

```
y = 2 * x + 3
```

```
# 3. Create the plot
```

```
plt.figure(figsize=(8, 5))
```

```
plt.plot(x, y, label='y = 2x + 3',
color='blue', linewidth=2)
```

```
# 4. Add labels, grid, and reference axes
```

```
plt.title('Plot of  $y = 2x + 3$ ', fontsize=14)
```

```
plt.xlabel('x', fontsize=12)
```

```
plt.ylabel('y', fontsize=12)
```

```
plt.axhline(0, color='black', linewidth=0.8, linestyle='--')
```

```
#x-axis line
```

```
plt.axvline(0, color='black', linewidth=0.8,          linestyle='--'
)
```

```
# y-axis line
```

```
plt.grid(True, linestyle=':', alpha=0.7)
```

```
plt.legend(fontsize=12)
```

```
# 5. Show the plot
```

```
plt.show()
```

Plotting with python

Matplotlib
– (basic) 

Low-level precision control. Ideal for exact layout customizations, custom multi-panel figures, and scientific graphics.

Seaborn 

High-level statistical library built on Matplotlib. Integration with Pandas DataFrames and elegant default visual themes.

**More-
Plotly**

Matplotlib Concepts

Pyplot Module: standard import

```
import matplotlib.pyplot as plt
```

provides a stateful interface for plotting.

Figure vs Axes: The Figure acts as the canvas; Axes represent individual subplot regions containing data elements.

Essential Charts: Create line plots with `plt.plot()` and scatter plots with `plt.scatter()` effortlessly.

Plot Annotations: Add clarity using `plt.title()`, `plt.xlabel()`, and `plt.legend()`.



Parts of a Matplotlib Figure

 **Figure Canvas:** Top-level container holding all axes, titles, legends, colorbars, and nested subplots.

 **Axes & Axis Objects:** The plotting space where data points live, managing ticks, limits, scale, and grid lines.

 **Custom Styling:** Configure markers, line styles, transparency (alpha), and colormaps for maximum visual impact.

 **High-Res Export:** Save graphics to PNG, SVG, or PDF formats using `plt.savefig('output.png', dpi=300)`.

Setup & Data Generation

```
import matplotlib.pyplot as plt  
import numpy as np
```

```
#1. Generate x values  
x = np.linspace(-10, 10, 100)
```

```
# 2. Define the function  
y = 2 * x + 3
```

What Happens Here?

- **Libraries:** pyplot gives us standard plotting routines, while numpy handles array calculations efficiently.
- **np.linspace(-10, 10, 100):** Creates an array of 100 numerical points equally spaced from -10 to +10.
- **Vectorized Operation:** $y = 2 * x + 3$ evaluates the formula across all 100 values simultaneously without needing a slow Python loop.

3. Create the plot

```
plt.figure(figsize=(8, 5))
```

```
plt.plot(x, y, label='y = 2x + 3', color='blue', linewidth=2)
```

Key Parameters Breakdown

- **plt.figure(figsize=(8, 5))**: Initializes a new figure canvas with an aspect dimension ratio of 8 inches wide by 5 inches tall.
- **plt.plot(x, y)**: Maps array x to the horizontal axis and array y to the vertical axis.
- **label='y = 2x + 3'**: Assigns a text label stored for the legend box.
- **color='blue', linewidth=2**: Sets the line color and increases thickness for readability.

4. Add labels, grid, and reference axes

```
plt.title('Plot of  $y = 2x + 3$ ', fontsize=14)
plt.xlabel('x', fontsize=12)
plt.ylabel('y', fontsize=12)
plt.axhline(0, color='black', linewidth=0.8, linestyle='--')
plt.axvline(0, color='black', linewidth=0.8, linestyle='--')
```

Adding Context & Reference Lines

- **plt.title() & Labels:** Clarify what dataset is being presented with custom font sizing.
- **plt.grid():** Draws a dashed background grid ('--') with 70% opacity (alpha=0.7) to help gauge values.
- **plt.axhline(0) & plt.axvline(0):** Draws thin black reference axes through (0,0) highlight origin intercepts.

Legend & Final Display



`plt.legend()`

Scans all active plot elements for defined label strings and automatically builds a clean legend box inside the plot bounds.



`plt.show()`

Displays the completed figure window or inline rendering in Jupyter Notebooks and frees memory resources.

Plotting

```
import matplotlib.pyplot as plt
import numpy as np
```

```
#1. Generate x values (100
#evenly spaced numbers from -
#10 to 10)
```

```
x = np.linspace(-10, 10, 100)
```

```
# 2. Define the function
```

```
# $y = 2x + 3$ 
```

```
y = 2 * x + 3
```

```
# 3. Create the plot
```

```
plt.figure(figsize=(8, 5))
```

```
plt.plot(x, y, label='y = 2x + 3',
color='blue', linewidth=2)
```

```
# 4. Add labels, grid, and reference axes
```

```
plt.title('Plot of  $y = 2x + 3$ ', fontsize=14)
```

```
plt.xlabel('x', fontsize=12)
```

```
plt.ylabel('y', fontsize=12)
```

```
plt.axhline(0, color='black', linewidth=0.8, linestyle='--')
```

```
#x-axis line
```

```
plt.axvline(0, color='black', linewidth=0.8,          linestyle='--'
)
```

```
# y-axis line
```

```
plt.grid(True, linestyle=':', alpha=0.7)
```

```
plt.legend(fontsize=12)
```

```
# 5. Show the plot
```

```
plt.show()
```

Numpy for functions. Plot a sinusoidal function

```
import numpy as np
import matplotlib.pyplot as plt
x = np.linspace(0, 10, 100)
y = np.sin(x)
plt.plot(x, y)
plt.xlabel('x')
plt.ylabel('sin(x)')
plt.title('Sine Wave')
plt.show()
```

Practice: Try changing the plot, change the type of line, color, add axis labels, change font size. Try plotting `np.cos(x)`

Eg-# Red dashed line ('r' for red, '--' for dashed)
`plt.plot(x, y, 'r--')`
Or green circles
`plt.plot(x, y, 'g:o')`

Drill 1.8

Plot $y = x ** 2$ or $y = \cos(x)$

Use. `np.cos(x)`

Choose the range of x values.

Random numbers

```
import random
```

```
random.uniform(0,1)
```

```
print(random.uniform(0,1))
```

```
num = random.randint(0,100)
```

```
print(num)
```

Drill 1.9

Write the Vibonacci series

Fibonacci series- 1,1,2,3,5,8,13 etc.

Each term is the sum of the previous two terms

In Vibonacci series- use a random number to decide if you want to add or subtract

Plot it as a function of iteration.

What is NumPy?



The ndarray Object

The core building block is the **N-dimensional array**. Unlike standard Python lists, all elements must share the same data type for maximum efficiency.



Blazing Fast Performance

Written under the hood in C, NumPy arrays use continuous memory layouts, performing mathematical operations up to 100x faster than raw Python loops.



Vectorized Operations

Apply mathematical functions across entire datasets at once (e.g., $y = 2 * x + 3$) without writing tedious for loops.

Arrays, Numpy arrays

```
import array as arr
# Create an array of integers
a = arr.array('i', [1, 2, 3])
# Access elements
print(a[0])
#Append an element
a.append(5)
print(a)
```

```
#Accessing
print(a[2])
#index starts at 0
```

Numpy arrays

```
import numpy as np
# create a numpy array from a list
arr = np.array([1,2,3])
```

```
#Multi-dimensional array
mat = np.array([[1,2,3],[4,5,6]])
print(mat)
print(mat.size)
print(mat.shape)
```

"""

Basic operations on
Numpy Arrays

"""

```
a = np.array([1, 2, 3])
b = np.array([4, 5, 6])
print(a + b) # [5 7 9]
print(a * 2) # [2 4 6]
print(a ** 2) # [1 4 9]
```

Zero Array, Empty Array

```
import numpy as np
```

```
arr = np.empty(5)
```

```
print(arr)
```

```
barr = np.zeros(5)
```

```
print(barr)
```

```
carr = np.zeros(5, dtype=int)
```

```
print(carr)
```

NumPy in Action

```
import numpy as np

# 1. Create 1D and 2D arrays
arr1 = np.array([10, 20, 30, 40])
arr2 = np.linspace(0, 1, 5)

# 2. Vectorized arithmetic
squared = arr1 ** 2

# 3. Built-in statistics
mean_val = np.mean(arr1)
max_val = np.max(arr1)
```

Key Operations Explained

- **np.array(...):** Converts a Python list into a high-performance ndarray.
- **np.linspace(start, stop, num):** Generates num evenly spaced numbers over a specified interval.
- **Element-wise Math:** `arr1 ** 2` squares every single number individually: [100, 400, 900, 1600].
- **Fast Aggregations:** Functions like `np.mean()` calculate summary stats across millions of values instantly.

Drill 1.10

- **Create a Python list** of numbers and print each element by index.
- **Make a NumPy array** from `[1][2][4]`, print its dtype, and add 1 to every element.
- **Create a 2D NumPy array** (3x3) filled with zeros, then fill its diagonal with ones.

Series vs. DataFrames

Pandas Series

A **1-Dimensional** labeled array capable of holding any data type (integers, strings, floats).

Think of it as a single column in a spreadsheet, accompanied by a explicit row index.

Pandas DataFrame

A **2-Dimensional** tabular data structure with labeled axes (rows and columns).

Think of it as an entire Excel spreadsheet or SQL table built right into Python.

Pandas in Action

```
import pandas as pd

# 1. Create a DataFrame from a dict
data = {
    'Name': ['Alice', 'Bob', 'Charlie'],
    'Age': [25, 30, 35],
    'Score': [88.5, 92.0, 95.5]
}
df = pd.DataFrame(data)

# 2. Filter rows conditionally
high_scores = df[df['Score'] > 90]
```

Step-by-Step Breakdown

- **Dict to DataFrame:** Keys become column headers ('Name', 'Age', 'Score'), and list values populate the columns.
- **Indexing:** Rows are automatically assigned integer index IDs starting at 0.
- **Expressive Filtering:** `df[df['Score'] > 90]` filters the table to return only records where Score exceeds 90 (Bob and Charlie).

NumPy vs. Pandas Comparison

Feature	NumPy	Pandas
Primary Object	ndarray (N-dimensional array)	DataFrame & Series
Data Types	Homogeneous (single type per array)	Heterogeneous (mixed column types)
Best Used For	Linear algebra, matrix math, raw computations	Tabular data analysis, CSV/SQL importing, cleaning
Indexing	Integer-based positions <code>arr[0, 1]</code>	Named columns & custom row labels <code>df['Score']</code>

Drill 1.11

Array Creation & Shapes

1. Create a 1D NumPy array with numbers from 10 to 49 (inclusive).
2. Reshape this array into a 5x8 matrix.
3. Print the array's **shape**, **data type**, and **total number of elements**.

```
# Drill 1.11
arr = np.arange(10, 50)
matrix = arr.reshape(5, 8)
print("Shape:", matrix.shape)
print("Dtype:", matrix.dtype)
print("Size:", matrix.size)
#Also try
print(matrix.shape[0])
print(matrix.shape[1])
```

Drill 1.12

Given the matrix:

```
import numpy as np
matrix = np.array([ [5, 10, 15, 20], [25, 30, 35, 40], [45, 50, 55, 60]])
```

- Extract the element 35.
- Extract the second column ([10, 30, 50]).
- Extract the bottom-right 2x2 sub-matrix ([[35, 40], [55, 60]]).

Drill 1.12

```
matrix = np.array([ [5, 10, 15, 20], [25, 30, 35, 40], [45, 50, 55, 60] ])
print("Element 35:", matrix[1, 2])
print("Second column:", matrix[:, 1])
print("Bottom-right 2x2:\n", matrix[1:, 2:])
```

Drill 1.13: Masking & Vectorization

1. Generate an array of 20 random integers between 1 and 100 using `np.random.randint()`.
2. Find all values greater than 50 using a boolean mask.
3. Replace all values less than or equal to 50 with -1.

```
# Drill 1.13
np.random.seed(42)
# For reproducible output
data = np.random.randint(1, 101, size=20)
print("Over 50:", data[data > 50])
data[data <= 50] = -1
print("Modified:", data)
# To convert array into numpy array
data=np.array(data)
```

Basic File Handling

A step-by-step introduction to reading text files, writing data, and loading multi-column files into Pandas DataFrames.

Common Access Modes:

Read Mode

Write Mode

Append Mode

Files

Syntax:

```
file_object = open('filename.txt', 'mode')
```

Mode – 'r', 'w', 'a'

Other modes are also available.

Reading a File in Python

```
# Method 1: Using 'with' (Recommended)
with open('sample.txt', 'r') as file:
    content = file.read()
    print(content)

# Method 2: Reading line by line
with open('sample.txt', 'r') as file:
    for line in file:
        print(line.strip())
```

Key Reading Concepts

open(filename, 'r'): Opens file in *Read* mode (`'r'`).

The with statement: Automatically closes the file when done, preventing resource leaks.

read(): Reads the entire file as a single string.

for line in file: Iterates line-by-line, ideal for memory efficiency on large files.

Writing & Appending to Files

```
# Overwriting / Creating a new file
with open('output.txt', 'w') as file:
    file.write("Hello, Python!\n")
    file.write("Writing line two.")

# Appending to an existing file
with open('output.txt', 'a') as file:
    file.write("\nAdding a new line!")
```

Write Modes Explained

Write Mode ('w'): Creates a new file or *completely overwrites* the existing file.

Append Mode ('a'): Adds new data to the *end* of the file without deleting existing text.

\n Character: Adds a new line break after your text string.

Automatic Creation: Both 'w' and 'a' create the file if it doesn't exist yet.

Read a single column file

```
def read_single_column_file(filepath, has_header=True):  
    with open(filepath, 'r', newline='') as file:  
        reader = csv.reader(file)  
        if has_header:  
            header = next(reader)  
            # Skip header  
        data = [row[0] for row in reader]  
    return data
```

Example usage: filepath = 'your_file.txt'

Set has_header to True if file has a header, False otherwise

data = read_single_column_file(filepath, has_header=True)

print(data)

Loading CSV Files into DataFrames

```
import pandas as pd

# Load a multi-column CSV file
df = pd.read_csv('students.csv')

# Inspect the dataset
print(df.head())

# Access specific columns
names = df['Name']
grades = df['Grade']
```

Why Use Pandas for Files?

Handles Headers: Automatically treats the first row as column titles (e.g., ID, Name, Grade).

Data Type Detection: Automatically parses numbers, strings, and dates into correct formats.

Multi-Column Support: Easily manages thousands of rows and columns with minimal memory overhead.

Handy read_csv() Parameters

Parameter	Example Usage	Description
sep or delimiter	<code>pd.read_csv('data.txt', sep='\t')</code>	Specify custom separators like tabs (\t) or semicolons (;).
usecols	<code>pd.read_csv('data.csv', usecols=['Name', 'Age'])</code>	Load only specific columns to save memory.
header	<code>pd.read_csv('data.csv', header=None)</code>	Use when file has no column names (assigns 0, 1, 2...).
index_col	<code>pd.read_csv('data.csv', index_col='ID')</code>	Set a specific column as the DataFrame index.

Exporting DataFrames to Files

```
import pandas as pd

# Sample DataFrame creation
data = {
    'Name': ['Alice', 'Bob'],
    'Age': [21, 22],
    'Major': ['CS', 'Math']
}
df = pd.DataFrame(data)

# Save back to a multi-column CSV
df.to_csv('output_data.csv', index=False)
```

Resources for further learning:

- NumPy official beginner guide
- GeeksforGeeks NumPy basics

Codecademy & W3Schools for hands-on tutorials